Implementation Plan — Spatial Degradation of Hepatocyte Zonation

Exact inputs, algorithms, formulas, and the code file for every step

Companion to the stubs in `src/steps/` and the reference implementation `src/pipeline.py`

Contents

This document expands each stub in `src/steps/` into exact inputs, algorithms, formulas, data shapes, outputs, and acceptance checks. It is the "fill-in-the-blanks" companion: the docstrings say *what*; this says *exactly how*. Reference implementations for the statistical steps already live in `src/pipeline.py` (the integrated donor-level script), and each step section below ends with a **Code:** line naming the file(s) that implement it.

1 Conventions used throughout

1.1 Paths and the config module

All paths are imported from `src/config.py`; no hard-coded paths appear in the steps.

- `PAPER1` — directory of the Paper 1 (disease atlas) inputs: `counts.mtx`, `genes.txt`, `barcodes.txt`, `cell_metadata.csv`, `metadata_all_cells.csv`.
- `PAPER2_TRAIN` — `paper2_train.npz`, the classifier training bundle.
- `SIGNATURES` — the `signatures/` directory of zonation gene lists.
- `FIGURES`, `TABLES` — output directories for artefact figures and CSVs.
- `config.signature_files(name)` — returns the `(pericentral, periportal)` file pair for a chosen signature set (see Section 1.5); `config.DEFAULT_SET` selects the default.

1.2 Matrix orientation, donor, and stage

- **Matrix.** `counts.mtx` is *genes* × *cells* (30,117×69,426), raw integer UMIs. Row i ↔ `genes.txt[i]`; column j ↔ `barcodes.txt[j]`.
- **Donor** = `metadata_all_cells.csv["Patient.ID"]` (about 47 donors).

- **Stage** = `cell_metadata.csv["stage"]`, an ordered factor mapped to ranks $0 \ldots 4$ by the helper `S2R`:

Healthy control < NAFLD < NASH w/o cirrhosis < NASH with cirrhosis < end stage $\longmapsto$ $0, 1, 2, 3, 4$.

1.3 The golden rule — donor is the unit of inference

The unit of inference is the **donor**, not the cell. For every hypothesis we aggregate cells into *one number per donor*, then run the statistical test on the ~ 47 donor values. Cell-level p-values are pseudoreplication and are never reported as the primary result.

1.4 The A4 table (the shared backbone)

Step 4 emits a per-cell table consumed by Steps 5–8 with columns

`cell_id, donor, stage, zonation_coord, pc, pp, plasticity, zone_p0, zone_p1, zone_p2, entropy`.

1.5 Signature sets — transcriptome-wide by default

The zonation coordinate is read from a **gene program**, and the size of that program is a first-class design choice. The motivation is to read position from the *whole* zonation program, not from a handful of marker genes.

Primary set: `full` (transcriptome-wide). The default signature is the full transcriptome-wide zonation program from Paper 2:

- `signatures/pericentral_full.txt` — 1,273 pericentral genes,
- `signatures/periportal_full.txt` — 364 periportal genes,

for a total of $\approx$ 1,637 *significantly-zonated* hepatocyte genes (Paper 2, $q < 0.05$, well-expressed, and present in Paper 1). This is `config.DEFAULT_SET = "full"`. **Both** the coordinate *and* the classifier use this full set. The classifier's `paper2_train.npz` cache now stores the *union of all tiers* ($\sim$1,700 genes), so any set can be sliced from it by gene name; re-run `src/prep/02_convert_paper2_mat.py` once if the cache predates this change.

Robustness / baseline alternatives. Smaller sets are kept as robustness and baseline checks, each selectable via `config.signature_files(name)`:

Set name	Size (PC / PP)	Role
`full`	1273/364	primary, transcriptome-wide (default)
`expanded`	$\sim$ 102/91	ranked top genes, $\sim$100 each
`core`	13/8	hand-curated canonical markers
`paper2_landmark`	20/20	EXACT Paper 2 hepatocyte landmark genes (verbatim, `Hepatocyte-*-LM.`

The decision: `full` is the result, `paper2_landmark` is the audit. We report both, but they do *different jobs* — this is not a 50/50 hedge. `full` is primary because (i) it *is* the contribution (read position from the entire program, not markers by eye); (ii) it is the *more robust* ruler in disease — the coordinate is a *mean* of z-scored genes, so disease-corruption of any single marker is diluted (e.g. `CYP2E1`, canonical pericentral, is induced in NASH for non-spatial reasons and could swing a 13-gene ruler, but is 1-of-∼1,600 here); and (iii) only a large set makes the H2 "which programs collapse" question and its non-circular held-out-gene test (Step 7) meaningful. `paper2_landmark` is the *credibility control*: the exact 20+20 hepatocyte landmark genes Paper 2 itself uses to assign zonation (verbatim from `Hepatocyte-*-LM.csv`), canonical and fully interpretable. If it shows the *same* collapse, the signal is not an artefact of gene-set size; if the two *disagree*, that is itself a finding (a warning the ruler may be disease-sensitive). **Per hypothesis:** H1 (collapse) + ruler/classifier → `full` primary, `paper2_landmark` control; H2 → `full` with the held-out repeated split; H3 → de-zonation score from `full`. `expanded`/`core` are an optional robustness gradient. **Why the original plan said "landmark":** marker-based reconstruction is the field-standard, validated method (Paper 2; Halpern et al. 2017) — the conservative starting point; `full` is the deliberate upgrade.

The PC/PP imbalance (1,273 vs 364) does not bias the coordinate. This is a *feature*-set asymmetry, not ML class imbalance (which concerns label counts, e.g. the classifier's zone terciles). The coordinate is $\text{mean}_z(\text{PC}) - \text{mean}_z(\text{PP})$; because each arm is a *mean* (not a sum), the 3.5:1 count ratio carries *no* magnitude bias. It does create a *precision* asymmetry — the PP arm averages fewer genes, so it is noisier and the periportal end is estimated less reliably. Mitigations: (1) standardise each arm to unit variance across cells *before* subtracting (equalises the arms regardless of count); (2) a size-matched run — top-364 strongest-zonated PC genes vs the 364 PP — as a robustness check; (3) optionally strength-weight genes by Paper 2 zonation effect size, which shrinks the imbalance by down-weighting the long tail of weak PC genes. The imbalance itself is part biology (zonation skews pericentral here) and part the selection thresholds ($q < 0.05$ + p90 expression + center-of-mass split).

Code: `src/config.py` (`PAPER1`, `PAPER2_TRAIN`, `SIGNATURES`, `FIGURES`, `TABLES`, `config.DEFAULT_SET`, `config.signature_files("full")`).

2 Step 1 — Build the ruler (prep)

2.1 Inputs

- Paper 2 landmark CSVs → `signatures/*_paper2_landmark.txt` (the 20+20 baseline).
- `supplementary_table_8.xlsx`, sheet `Hepatocyte` → `signatures/*_expanded.txt` (ranked, ∼ 100 each), and the full $q < 0.05$ program → `signatures/*_full.txt`.
- `combined_scRNAseq_atlas_M5M6M7M8.mat` → `paper2_train.npz`.

2.2 Training labels (the classifier's y)

Each Paper 2 nucleus receives a zone label:

- **Ground truth (target).** Port `parse_snRNAseq_combined_atlas.m`: Paper 2 mapped its *spatial* (Visium) zonation onto its snRNA nuclei; use that zone index, binned to 3 classes.

- **Fallback (current `paper2_train.npz`).** Compute on Paper 2's own hepatocytes

$$\text{zone\_label} = \text{tercile}\big(\text{score(PC)} - \text{score(PP)}\big) \in \{0, 1, 2\}$$

(0 portal, 1 mid, 2 central).

2.3 Output and acceptance

Artefact A1. Acceptance: 3 zone classes present with balanced-ish counts; PC genes rise and PP genes fall along the zone index (profile plot).

Code: `src/prep/02_convert_paper2_mat.py`, `src/prep/03_build_signatures.py`, the `signatures/` directory; profile plot `src/plotting/artefacts.py:plot_a1_reference_profiles`.

3 Step 2 — Load & QC

3.1 Inputs

`PAPER1/{counts.mtx, genes.txt, barcodes.txt, cell_metadata.csv, metadata_all_cells.csv}`.

3.2 Algorithm

1. `M = scipy.io.mmread(PAPER1/"counts.mtx").tocsc()` → genes × cells sparse.
2. `genes = [l.strip() ...]`, `bars = [l.strip() ...]` (order = matrix rows / columns).
3. `meta = read_csv(cell_metadata).set_index("cell_id").reindex(bars)` → `stage = meta["stage"]`.
4. `allm = read_csv(metadata_all_cells).set_index("cell_id")`; auto-detect the donor column from `["Patient.ID","patient","donor","orig.ident","sample"]`; `donor = allm[col].reindex(bars)`.
5. `libsize = asarray(M.sum(0)).ravel()` (per-cell total UMIs).
6. **QC sanity only (do *not* re-filter).** Confirm the kept cells are hepatocytes — mean `ALB` expression is high; print per-stage and per-donor counts. Optional sensitivity: flag the lowest-`nCount_RNA` decile for a robustness re-run, do not delete.

3.3 Normalization (used by scoring; the `zrows()` helper)

For a gene row x, first CP10k then $\log 1p$:

$$v = \log\Big(1 + \frac{x}{\text{libsize}} \cdot 10^4\Big), \tag{1}$$

$$z = \frac{v - \text{mean}(v)}{\text{std}(v)} \qquad (\text{if } \text{std}(v) = 0,\ z = 0). \tag{2}$$

3.4 Output and acceptance

Output A2: `M, genes, bars, stage, donor, libsize` plus a per-stage / per-donor count table.

Acceptance: 69,426 hepatocytes; 5 stages with counts 3750/7073/31903/3603/23097; `ALB` clearly expressed.

Code: `src/steps/step2_load_qc.py` (ref `src/pipeline.py:load`).

4 Step 3 — Harmonize genes

4.1 Inputs

A2 `genes` plus the signature gene sets (and `paper2_train.feats` for the classifier).

4.2 Algorithm

1. Reconcile symbols (and Ensembl IDs where present) across Paper 1, Paper 2, and the signatures; report the mapping rate |shared|/|signature|.
2. Restrict signatures and classifier features to the shared set. If any signature drops below $\sim$15 genes, widen it via the `expanded` list.
3. Choose batch-robust features: per-gene z-score (above) and/or a rank transform, so the score measures portal-vs-central rather than snRNA-vs-Visium platform.

4.3 Output and acceptance

Output A3: a harmonization report (mapping rate, surviving signature sizes) plus the frozen gene sets. **Acceptance:** $\geq 80\%$ of signature genes survive the intersection.

Code: `src/steps/step3_harmonize.py`.

5 Step 4 — Place every cell

5.1 Step 4a — Signature scoring

For a gene set S, use the **background-subtracted score** (Scanpy `score_genes` style). Y is the z-scored log $1p$-CP10k expression: for gene g, cell c with raw count x_{gc} and library size $\ell_c = \sum_{g'} x_{g'c}$,

$$v_{gc} = \log\Big(1 + 10^4 \, \tfrac{x_{gc}}{\ell_c}\Big), \qquad Y_{gc} = \frac{v_{gc} - \mathrm{mean}_{c'}\, v_{gc'}}{\mathrm{std}_{c'}\, v_{gc'}}$$

(CP10k $\to$ log $1p$ $\to$ per-gene z-score across cells; the same normalization as Step 2, §3.3). Then for cell c,

$$\mathrm{score}_S[c] \;=\; \frac{1}{|S|}\sum_{g\in S} Y[g,c] \;-\; \frac{1}{|B|}\sum_{g\in B} Y[g,c],$$

where B is a control set drawn to match the expression-bin distribution of S. Subtracting B removes each cell's overall expression / depth baseline, so the score reflects S specifically.

Then, with each arm standardised to unit variance across cells (so the PC/PP gene-count imbalance does not tilt the axis; Section 1.5):

$$\mathrm{pc} = \mathrm{score}(\mathrm{PERICENTRAL}), \qquad \mathrm{pp} = \mathrm{score}(\mathrm{PERIPORTAL}), \tag{3}$$

$$\mathrm{coord} = \mathrm{pc} - \mathrm{pp} \;\; (>0 \text{ pericentral}), \qquad \mathrm{plast} = \mathrm{score}(\mathrm{PLASTICITY}). \tag{4}$$

The plasticity set is `signatures/plasticity.txt` (`KRT7, KRT19, SOX9, SOX4, KRT23, NCAM1`). **Run this for every set in** `config.SETS_TO_COMPARE` — the coordinate is produced for `full` *and* `paper2_landmark`, so Step 5 can pick the leader from the data. The default gene set is the `full` program (Section 1.5).

The simpler `src/pipeline.py:score` uses the mean of z-scored signature genes *without* an explicit control set; the `score_genes` background form above is the upgrade.

Code: `src/steps/step4_score.py` (ref `src/pipeline.py:score`); plot `src/plotting/artefacts.py:plot_a4_coordina`

5.2 Step 4b — Classifier + entropy

1. Train on `paper2_train.npz`: X (nuclei × feature genes, CP-normalized) and y = zone_label $\in \{0, 1, 2\}$. With the `full` set the features are the full zonated gene set (*not* the 40 landmarks).
2. `Pipeline(StandardScaler(), LogisticRegression(multi_class="multinomial", C=..., max_iter=...))`. Prefer a **calibrated** logistic model (well-behaved probabilities) over a random forest.
3. **Evaluate on a held-out Paper 2 split FIRST** (`train_test_split`, stratified by donor): accuracy and confusion matrix; must clearly beat chance (1/3).
4. Apply to Paper 1 (same features, same normalization): `P = clf.predict_proba(X1)` → `zone_p0..2`.
5. **Entropy** per cell:
$$H[c] = -\sum_k p_k \log p_k$$
(k over the 3 zones, natural log, range $[0, \log 3]$). Low H = confident zone; high H = de-zonated / confused.
6. **4c agreement.** `spearman(zonation_coord, expected_zone)` on Paper 1 (especially healthy) should exceed ~ 0.5.

Output A4: merge `zone_p0..2, entropy` into the per-cell table.

Code: `src/steps/step4b_classifier.py` (ref `src/classifier.py`).

6 Step 5 — Validate on healthy (GATE)

6.1 Input and algorithm

Input: A4 restricted to Paper 1 healthy donors. For each validation marker,

$$\rho = \text{spearman}\left(\text{coord}[h],\ \text{expr}_{\text{marker}}[h]\right).$$

- Pericentral `CYP2E1, CYP1A2, GLUL, PCK2` → expect $\rho > 0$.
- Periportal `ASS1, ALDOB, PCK1, HAL` → expect $\rho < 0$.
- Classifier entropy in healthy should be low (peaked predictions).

6.2 Output and the gate

Output A5: a bar of ρ per marker plus the values. **Acceptance / GATE:** pericentral $\rho > 0$ and periportal $\rho < 0$. *Do not start Step 6 until this passes.* If it fails: widen to `expanded` signatures, use rank features, revisit normalization.

Run the gate for *both* sets — and let it pick the leader. Compute Step 5 (and the Step 5b ruler battery) for `full` *and* `paper2_landmark`. This is where the "`full` vs landmark" choice is actually settled, *by evidence rather than by assertion*: lead with whichever set validates more cleanly on the healthy positive control (marker ρ signs + magnitudes, low entropy, strong PC–PP anti-correlation, high split-half reproducibility), and report the other alongside as the control. Honest expectation: `full` usually wins on resolution and is required for H2's held-out test; `paper2_landmark` may transfer more cleanly across the snRNA→Visium platform gap (fewer, canonical, high-expression genes carry less cross-dataset technical variance). If the two *disagree* on the collapse, that disagreement is a finding, not a nuisance — it flags the ruler as disease/platform-sensitive and must be reported.

Code: `src/steps/step5_validate.py` (ref `src/pipeline.py:validate`); plot `src/plotting/artefacts.py:plot_a5_va`

7 Step 5b — Is the ruler still a ruler?

Per stage s, on the cells of that stage:

1. **Internal coherence.** Mean pairwise correlation among pericentral genes (and among periportal genes). Healthy = high; if it crashes, the module is dissolving (not just de-zonating).
2. **Cross-program anti-correlation.** $\mathrm{corr}(\mathrm{pc}, \mathrm{pp})$ across cells. Healthy $\approx$ strongly negative; weakening toward 0 = true de-zonation (robust to global shifts).
3. **Split-half reproducibility.** Randomly split each signature in two, build two coordinates, and correlate them. A drop means the signature is no longer self-consistent.
4. **Program-off vs restriction-lost.** Track each module's mean magnitude (is it switched off?) versus the rate of cells co-expressing both ends (is positional restriction lost?).

Output A5b: per-stage curves; tells you *which kind* of breakdown each stage is.

Code: `src/steps/step5b_ruler_diagnostics.py`; plot `src/plotting/artefacts.py:plot_a5b_ruler`.

8 Step 6 — Collapse curve (H1)

8.1 Per-donor metrics

For donors with ≥ 30 cells, one row per donor:

$$\mathrm{spread}[d] = \mathrm{std}\big(\mathrm{coord}[\mathrm{donor} = d]\big) \quad \text{(narrows with disease)}, \tag{5}$$

$$\mathrm{anticorr}[d] = \mathrm{spearman}\big(\mathrm{pc}[\mathrm{donor} = d],\ \mathrm{pp}[\mathrm{donor} = d]\big) \quad \text{(rises toward 0)}, \tag{6}$$

$$\mathrm{entropy}[d] = \mathrm{mean}\big(H[\mathrm{donor} = d]\big) \quad \text{(rises, if classifier run)}, \tag{7}$$

$$\mathrm{stage\_rank}[d] = \mathrm{S2R}\big[\,\mathrm{mode}(\mathrm{stage}[\mathrm{donor} = d])\big]. \tag{8}$$

8.2 Ordered-trend test on the ~ 47 donor values

- **Spearman.** `rho, p = spearmanr(stage_rank, metric)`; with no ties

$$\rho = 1 - \frac{6 \sum_d d^2}{n(n^2 - 1)}.$$

- **Jonckheere–Terpstra** (ordered alternative across the 5 stage groups):

$$J = \sum_{a<b} U(\text{group}_a, \text{group}_b),$$

where U counts pairs with $x_b > x_a$. Standardize with the null mean

$$\mu_J = \frac{N^2 - \sum_s n_s^2}{4}$$

and the standard JT variance σ_J^2 to obtain z and p.

8.3 Confidence and negative control

- **Donor bootstrap CI.** Resample donors with replacement $B = 2000$ times, recompute ρ, and take the 2.5/97.5 percentiles.
- **Negative control (permutation).** Permute `stage_rank` across donors $B = 2000$ times, recompute $|\rho|$, and report

$$p_{\text{perm}} = \frac{\#\{\text{ null} \geq \text{observed}\} + 1}{B + 1}.$$

Output A6: `collapse_per_donor.csv` plus a plot (donor points + per-stage mean) and per-metric ρ, 95% CI, p, p_{perm}. **Acceptance:** spread ↓, anticorr → 0, (entropy ↑); CI excludes 0; p_{perm} small.

Code: `src/steps/step6_collapse.py` (ref `src/pipeline.py:collapse`); plot `src/plotting/artefacts.py:plot_a6_co`

9 Step 7 — Zone-resolved DE (H2)

9.1 Bin cells into zones

By coordinate terciles: `terc = quantile(coord, [1/3, 2/3])`, `zone = digitize(coord, terc)` → 0 portal, 1 mid, 2 central.

9.2 Pseudobulk (the donor-level move)

For each zone $z \in \{\text{portal}, \text{central}\}$, for each donor d with ≥ 20 cells in that zone, build one profile:

$$\text{cells} = (\text{zone} = z)\ \&\ (\text{donor} = d), \tag{9}$$

$$\text{cp} = \text{per-cell CP-normalized counts} = \text{counts/colsum}, \tag{10}$$

$$\text{PB}[d, :] = \log1p\Big(\text{mean}_{\text{cells}}\,(\text{cp}) \cdot 10^4\Big) \qquad (\text{donor} \times \text{gene}), \tag{11}$$

$$\text{ranks}[d] = \text{stage\_rank}[d]. \tag{12}$$

9.3 Test per gene (detected in $\geq 10\%$ of the zone's donors)

- **MVP.** `rho, p = spearman(PB[:,g], ranks)` across donors.
- **Cleaner H2 (the real test).** OLS interaction on the pseudobulk

$$\text{expr}_g \ \sim\ \text{coord} + \text{stage} + \text{coord:stage};$$

the coord:stage coefficient answers "does this gene's zonal slope weaken with stage?" — i.e. lost zonation, not just lost level.

9.4 Multiple testing (BH-FDR)

Sort the p-values ascending; the threshold is $p_{(k)} \leq \frac{k}{m}\alpha$; take the largest such k; everything below it is significant; the q-value is the monotone-adjusted p. Report at $q < 0.05$.

9.5 Circularity guards (mandatory)

The coordinate is built *from* the signature genes, so testing those same genes for "collapse" is circular. Two guards:

- **Signature flag.** Flag `is_signature` genes and report significant counts BOTH including and EXCLUDING them; "driver" claims use non-signature genes.
- **Held-out gene split (the key fix).** Build the coordinate on one random half of the gene set and test collapse on the disjoint other half.

Repeated random held-out splits. Because any single random partition could be lucky, repeat it K times ($K \approx 20$–50 random splits) and report the *distribution* of the result across splits:

- the **median effect** across splits, and
- the **fraction of splits** that are significant.

A result that holds across most random splits is robust; one that depends on a particular partition is not. Practical notes:

- **Fix the RNG seed** for reproducibility.
- **Optionally stratify** the split so each half keeps both PC and PP genes.
- This is **meaningful only with the large `full` set**, not 20 landmarks — you cannot split 40 genes into two informative halves.

Output A7: `de_portal.csv`, `de_central.csv` (`gene, effect, p, q, is_signature`) plus a volcano plot. **Acceptance:** the significant set survives excluding signature genes *and* holds across most random splits.

Code: `src/steps/step7_de.py` (ref `src/pipeline.py:de`); plot `src/plotting/artefacts.py:plot_a7_volcano`.

10 Step 8 — De-zonation ↔ plasticity (H3)

10.1 De-zonation proxy

Per cell, distance of coord from the committed ends:

$$\text{dez} = -\left|\frac{\text{coord} - \text{median(coord)}}{\text{std(coord)}}\right|$$

(near the middle ⇒ more de-zonated); high entropy is an alternative proxy.

10.2 The confound and the within-donor test

Both `dez` and `plasticity` rise with stage, so a *pooled* correlation is a fake-positive. Test **within donor**, then aggregate:

- Per donor d ($\geq$ 30 cells): $r[d]$ = spearman $\big($dez[donor $= d$], plasticity[donor $= d$]$\big)$; report mean(r) and the % of donors with $r > 0$.
- **Regression (formal version).** OLS plasticity $\sim$ dez$+C$(stage)$+C$(donor); the `dez` coefficient is the association *after* removing stage and donor; report its sign and p.
- **Optional two-group view.** MannWhitneyU(plasticity[de-zonated], plasticity[zonated]) within stage, with a rank-biserial effect size.

Output A8: a stage-stratified scatter plus the within-donor statistic and the `dez` coefficient. **Acceptance:** the within-donor (not pooled) effect is reported with sign and p, honestly (positive or null).

Code: `src/steps/step8_plasticity.py` (ref `src/pipeline.py:plasticity`); plot `src/plotting/artefacts.py:plot_a`

11 Step 9 — Bonus: collapse vs inherited risk

11.1 Inputs

- `hits` = the Step-7 hit list ($q < 0.05$, non-signature).
- `risk` = Paper 3 risk-variant target genes.
- background = genes **testable** in Step 7 (detected + tested), ideally expression-matched.

11.2 Test (hypergeometric enrichment)

With $k = |\text{hits} \cap \text{risk}|$, $n = |\text{hits}|$, $K = |\text{background} \cap \text{risk}|$, $N = |\text{background}|$:

$$p = \text{hypergeom.sf}(k-1,\ N,\ K,\ n) \qquad \text{(scipy)}, \tag{13}$$

$$\text{fold} = \frac{k/n}{K/N}, \tag{14}$$

with BH applied across the few tests.

Output A9: fold + CI + q. **Acceptance:** the background is well-defined and the result is stated with caveats (associational). Strictly optional.

Code: `src/steps/step9_bonus_enrichment.py`.

12 Plotting machinery

Function	Input	Draws
`plot_a1_reference_profiles`	gene × zone means	PC rising / PP falling along zone
`plot_a4_coordinate`	A4 table	coord histogram + 2 marker scatters
`plot_a5_validation`	A4 healthy + markers	bar of ρ, colored by expected sign
`plot_a5b_ruler`	per-stage diagnostics	coherence / anti-corr / split-half curves
`plot_a6_collapse`	per-donor metrics	donor points + per-stage mean
`plot_a7_volcano`	DE table	effect vs $-\log_{10} q$ volcano
`plot_a8_plasticity`	A4 table	stage-stratified dez-vs-plasticity scatter

Code: `src/plotting/artefacts.py`.

13 Drivers

- **Full pipeline driver.** `src/run_all.py` — runs Steps 2–9 end to end and writes all artefacts.
- **Positive control.** `src/run_p2_validation.py` — runs the ruler against Paper 2's own data, where the answer is known, as a sanity check on scoring and the classifier.

Code: `src/run_all.py`, `src/run_p2_validation.py`.